COMPLEXIDADE DE ALGORITMOS

Análise de Algoritmos: Caso Médio, Pior Caso e Amortizada

Felipe A. Przysiada

REVISÃO

Por que Medir o Tempo em Função do Tamanho da Entrada?

REVISÃO

Motivação

1. O tempo real (em segundos) depende do hardware, da linguagem e do compilador — não é comparável entre execuções diferentes.
2. O número de operações executadas, em função do tamanho n da entrada, é uma propriedade do algoritmo, não da máquina.
3. Medir em função de n permite prever o comportamento do algoritmo para entradas maiores — essencial para planejamento de capacidade.
4. É a base que permite comparar dois algoritmos de forma justa, independentemente de onde forem executados.

REVISÃO

Mesmo n, Tempos Diferentes

Para um mesmo tamanho de entrada n, o número de operações executadas pode variar bastante — depende do conteúdo da entrada, não apenas do seu tamanho.

> *Exemplo: buscar um valor em um vetor de n elementos pode custar 1 comparação (se estiver na primeira posição) ou n comparações (se estiver na última, ou não existir).*

Isso motiva perguntas diferentes: qual é o comportamento no melhor caso? No pior caso? Em média? E ao longo de uma sequência de operações?

REVISÃO

Quatro Lentes de Análise

Melhor caso	o mínimo de operações, sobre todas as entradas de tamanho n
Pior caso	o máximo de operações, sobre todas as entradas de tamanho n
Caso médio	o valor esperado, assumindo uma distribuição sobre as entradas
Amortizada	o custo médio por operação ao longo de uma sequência de operações, garantido no pior caso da sequência

Análise de Pior Caso

Definição

Considera-se, entre todas as entradas possíveis de tamanho n, aquela que exige o maior número de operações.

$$T(n) = \max \{ \text{custo}(e) : e \text{ é uma entrada de tamanho } n \}$$

É a análise mais usada na prática: fornece uma **garantia** — o algoritmo nunca será mais lento do que T(n), não importa qual seja a entrada.

Exemplo: Busca Sequencial

Procurar um valor x em um vetor de n elementos, comparando um a um, do início ao fim.

Pior caso: x está na última posição, ou x não está presente no vetor. É necessário examinar todos os n elementos.

$$T(n) = n \rightarrow \Theta(n)$$

Exemplo: Quicksort

O Quicksort particiona o vetor em torno de um pivô e ordena recursivamente as duas partes.

Pior caso: o pivô escolhido é sempre o menor ou o maior elemento (ex.: vetor já ordenado, com pivô fixo no primeiro ou último elemento). Cada partição gera uma parte vazia e outra com $n-1$ elementos.

$$T(n) = T(n-1) + \Theta(n)$$

$$T(n) = \Theta(n^2)$$

(a soma 1+2+...+n das partições desbalanceadas gera o termo quadrático)

Por que é a Análise Padrão?

1. Fornece uma garantia, um limite superior válido para qualquer entrada, inclusive as adversariais.
2. Não depende de suposições sobre como as entradas são geradas.
3. É essencial em sistemas com restrições de tempo real, onde exceder o limite não é aceitável.
4. É, em geral, mais simples de calcular do que o caso médio, que exige um modelo probabilístico.

Exemplo Aprofundado: Insertion Sort

EXEMPLO APROFUNDADO

O Problema da Ordenação

Como problema de referência para introduzir melhor e pior caso, vamos analisar em detalhe um algoritmo clássico: o Insertion Sort.

```
Entrada: A[1..n]   →   Saída: A[1..n] em ordem crescente
```

O tamanho da entrada é a dimensão n do vetor, o algoritmo rearranja seus elementos em ordem crescente, ignorando os valores específicos que ele contém.

EXEMPLO APROFUNDADO

Insertion Sort - Ideia

A cada passo j (de 2 até n), o subvetor A[1..j−1] já está ordenado. O algoritmo toma chave = A[j] e desloca os elementos maiores de A[1..j−1] uma posição à direita, até achar a posição correta para a chave.

> *Exemplo: A = [20, 25, 35, 40, 44, 55, 38, 99, 10, 65, 50], com chave = A[7] = 38. Os elementos 55, 44, 40, 35 (maiores que 38) são deslocados uma posição à direita, e 38 é inserido entre 35 e 40.*

Ao final de cada passo j, o subvetor A[1..j] está ordenado, por indução, ao final do laço (j = n), o vetor inteiro A[1..n] está ordenado.

EXEMPLO APROFUNDADO

Insertion Sort - Pseudocódigo

```
INSERTIONSORT(A, n)
1  para j ← 2 até n faça
2       chave ← A[j]
3       # J vem aqui
4       i ← j - 1
5       enquanto i ≥ 1  e  A[i] > chave faça
6           A[i+1] ← A[i]
7           i ← i - 1
8       A[i+1] ← chave
```

EXEMPLO APROFUNDADO

Análise do Algoritmo

1. Finitude: o algoritmo pára? O Insertion Sort tem dois laços limitados (j e i), então sempre termina.

2. Corretude: o algoritmo faz o que promete? Por indução em j, ao final do laço A[1..n] está ordenado.

3. Complexidade de tempo: quantas instruções básicas são necessárias, no pior caso, para ordenar os n elementos?

EXEMPLO APROFUNDADO

Complexidade de Tempo

O tempo de execução (complexidade de tempo) é dado em função do tamanho da entrada. Na ordenação, o tamanho é a dimensão n do vetor — os valores dos elementos não importam.

Complexidade de tempo é o número de instruções básicas — operações elementares ou primitivas, como comparações, atribuições e operações aritméticas — executadas a partir de uma entrada.

EXEMPLO APROFUNDADO

Contando as Instruções Básicas

ck	custo (tempo) de cada execução da linha k do pseudocódigo
tj	número de vezes que a linha 5 (o teste do enquanto) é executada, para aquele valor de j

Na próxima página, usamos ck e tj para montar uma tabela de custo por linha do pseudocódigo do Insertion Sort.

EXEMPLO APROFUNDADO

Custo do Insertion Sort

Linha do INSERTIONSORT	Custo	Vezes
1 para j ← 2 até n faça	**c1**	n
2 chave ← A[j]	**c2**	n − 1
3 ▸ comentário	**0**	n − 1
4 i ← j − 1	**c4**	n − 1
5 enquanto i ≥ 1 e A[i] > chave	**c5**	Σtj
6 A[i+1] ← A[i]	**c6**	Σ(tj−1)
7 i ← i − 1	**c7**	Σ(tj−1)
8 A[i+1] ← chave	**c8**	n − 1

ck: custo de cada execução da linha k. tj: número de vezes que a linha 5 executa para o valor j.

EXEMPLO APROFUNDADO

Tempo Total T(n)

O tempo total T(n) é a soma dos tempos de cada linha, ponderados pelo número de vezes que cada linha é executada:

$$T(n) = c1n + c2(n-1) + c4(n-1) + c5 \cdot \Sigma tj + c6 \cdot \Sigma(tj-1) + c7 \cdot \Sigma(tj-1) + c8(n-1)$$

(cada tj é o número de vezes que a linha 5 executa ao inserir A[j] em A[1..j−1])

Entradas de tamanho igual (mesmo n) podem ter tempos de execução diferentes, pois T(n) depende dos valores tj.

Por isso, analisamos casos específicos e bem definidos: o melhor caso e o pior caso.

EXEMPLO APROFUNDADO

Melhor Caso

O melhor caso ocorre quando o vetor A já está ordenado. Assim, tj = 1 para todo j — a linha 5 nunca desloca elementos (linhas 6 e 7 não executam):

$$T(n) = c1n + c2(n-1) + c4(n-1) + c5(n-1) + c8(n-1)$$

(uma função da forma an + b)

$$T(n) = \Theta(n)$$

No melhor caso, o tempo de execução do Insertion Sort é uma função linear no tamanho da entrada.

EXEMPLO APROFUNDADO

Pior Caso

O pior caso ocorre quando A está em ordem decrescente: para inserir a chave, é preciso compará-la com todos os j−1 elementos anteriores, logo, tj = j:

$$T(n) = (c5+c6+c7)/2 \cdot n^2 + Bn + C$$

(as somas $\Sigma j = n(n+1)/2 - 1$ e $\Sigma(j-1) = n(n-1)/2$ geram o termo quadrático)

$$T(n) = \Theta(n^2)$$

No pior caso, o tempo de execução do Insertion Sort é uma função quadrática no tamanho da entrada.

EXEMPLO APROFUNDADO

Complexidade Assintótica

No pior caso, estudamos o comportamento de forma assintótica, "jogando para debaixo do tapete" as constantes e os termos de baixa ordem — $3n^2+10n+50$ se aproxima de $3n^2$ quando n cresce:

n	$3n^2+10n+50$	$3n^2$
64	12.978	12.288
512	791.602	786.432
4.096	50.372.658	50.331.648
16.384	805.470.258	805.306.368
32.768	3.221.553.202	3.221.225.472

$3n^2$ é o termo dominante quando n é grande.

Conclusão

Grosseiramente falando, podemos nos concentrar nos termos dominantes de T(n) e esquecer os demais — constantes e termos de baixa ordem não importam quando n é grande.

$$T(n) = \Theta(n^2)$$

Usando notação assintótica, dizemos que o Insertion Sort tem complexidade de tempo de pior caso $\Theta(n^2)$ — e complexidade de melhor caso $\Theta(n)$.

Análise de Caso Médio

Definição

Considera-se o valor esperado do custo, assumindo uma distribuição de probabilidade sobre as entradas de tamanho n.

$$E[T(n)] = \Sigma_e \, P(e) \cdot custo(e), \quad \text{para toda entrada } e \text{ de tamanho } n$$

Diferente do pior caso, o resultado depende diretamente de qual distribuição P é assumida sobre as entradas.

Pressupostos sobre a Distribuição

A análise de caso médio exige um modelo de como as entradas ocorrem na prática. O pressuposto mais comum é o de distribuição uniforme.

> *Exemplo de pressuposto: todas as n! permutações de um vetor são igualmente prováveis, ou o elemento buscado tem a mesma chance de estar em qualquer posição.*

- Nem sempre é realista: dados reais costumam ter padrões (quase ordenados, repetidos, correlacionados).
- Um pressuposto errado invalida a conclusão — o caso médio não é uma propriedade só do algoritmo, mas do algoritmo + modelo de entrada.

Exemplo: Busca Sequencial

Pressuposto: o valor buscado está presente no vetor, e tem a mesma probabilidade 1/n de estar em cada uma das n posições.

$$E[T(n)] = (1/n) \cdot \sum_{i=1}^{n} i = (n+1)/2$$

Resultado: em média, cerca de metade do vetor é examinada — **$\Theta(n)$**, mas com constante menor do que o pior caso.

Exemplo: Quicksort

Pressuposto: todas as n! permutações da entrada são igualmente prováveis (ou o pivô é escolhido aleatoriamente).

Em média, as partições ficam razoavelmente balanceadas ao longo das chamadas recursivas, o que leva à recorrência:

$$T(n) = 2T(n/2) + \Theta(n)$$

$$T(n) = \Theta(n \log n)$$

Muito melhor que o pior caso $\Theta(n^2)$, por isso o Quicksort é, na prática, um dos algoritmos de ordenação mais usados.

Análise Amortizada

Conceito Intuitivo

Mede o custo médio por operação ao longo de uma **sequência** de n operações — mas, diferente do caso médio, essa garantia vale para ***qualquer*** sequência possível, sem suposição de probabilidade.

Algumas operações da sequência podem ser caras (O(n)), desde que sejam raras o suficiente para que o custo total da sequência inteira permaneça baixo.

Amortizada × Caso Médio

Caso médio	probabilística — depende de um pressuposto sobre a distribuição das entradas
Amortizada	determinística — garantia de pior caso sobre a sequência de operações, sem suposições

Por isso a análise amortizada é tão usada em estruturas de dados: dá uma garantia robusta, válida mesmo contra um adversário que escolhe a pior sequência de operações possível.

Motivação: Vetor Dinâmico

Um vetor dinâmico (ex.: ArrayList, std::vector) cresce automaticamente: quando fica cheio, dobra de capacidade e copia todos os elementos para o novo espaço.

- A maioria das inserções (push_back) apenas grava no espaço já disponível: custo O(1).
- Quando o vetor está cheio, a inserção dispara um redimensionamento: aloca o dobro do espaço e copia todos os elementos, custo O(n).

Pergunta: qual é o custo, no pior caso, de uma sequência de n inserções?

Custo das Inserções no Vetor Dinâmico

Ao inserir o elemento de número i (1-indexado), há redimensionamento exatamente quando i é uma potência de 2 (1, 2, 4, 8, ...), copiando os $i-1$ elementos existentes.

Pior caso de uma única inserção: O(n) — quando ocorre o redimensionamento.

Mas essa inserção cara é rara: entre n inserções, o redimensionamento ocorre apenas cerca de $\log_2 n$ vezes. Isso sugere que o custo médio por operação é bem menor que O(n) — vamos provar isso formalmente com o método do agregado.

Método do Agregado (Aggregate Method)

Definição

Soma-se o custo total de uma sequência de n operações no pior caso e divide-se pelo número de operações.

```
custo amortizado = T(n) / n
```

É o método mais simples de análise amortizada: não distingue o custo entre operações diferentes, apenas garante o total ao longo da sequência.

Aplicação: Vetor Dinâmico

Custo total de n inserções: o custo fixo de cada inserção (1) mais o custo das cópias nos redimensionamentos (1, 2, 4, ..., até n).

$$T(n) \leq n + (1 + 2 + 4 + \ldots + n) \leq n + 2n = 3n$$

(soma geométrica de razão 2: 1+2+4+...+n < 2n)

Custo amortizado por inserção = T(n)/n ≤ 3n/n = O(1)

Mesmo que uma inserção isolada custe O(n) no pior caso, o custo médio por operação ao longo de qualquer sequência é constante.

Nota: Outros Métodos

O método do agregado é o mais direto, mas existem outras duas técnicas clássicas de análise amortizada:

Método da contabilidade (accounting method)
atribui um "custo amortizado" a cada operação, criando um crédito que paga pelas operações caras futuras.

Método do potencial (potential method)
define uma função Φ sobre o estado da estrutura, medindo o "crédito acumulado" a cada passo.

Ambos ficam fora do escopo de hoje, mas valem a pena para um próximo aprofundamento.

Exercício

EXERCÍCIO GUIADO

Enunciado

Considere uma pilha (stack) com três operações:

`PUSH(x)`	empilha o elemento x - custo O(1)
`POP()`	desempilha um elemento - custo O(1)
`MULTIPOP(k)`	remove min(k, tamanho atual) elementos do topo, um a um - custo O(min(k, tamanho))

Compare o pior caso, o caso médio e a análise amortizada para uma sequência de n operações, começando com a pilha vazia.

EXERCÍCIO GUIADO

Passo 1 - Pior Caso de uma Operação

Uma única chamada de **`MULTIPOP(k)`** pode remover até n elementos, se a pilha estiver cheia e $k \geq n$.

`Pior caso: Θ(n)`

Esse limite é real e correto — mas, se aplicado ingenuamente a cada uma das n operações da sequência, sugeriria um total de $O(n^2)$, o que é uma superestimativa grosseira.

EXERCÍCIO GUIADO

Passo 2 - Caso Médio

Pressuposto: cada operação da sequência é escolhida uniformemente entre PUSH e POP (ignorando MULTIPOP por simplicidade), cada uma com custo O(1).

$$E[\text{custo}] = \tfrac{1}{2}\cdot O(1) + \tfrac{1}{2}\cdot O(1) = O(1)$$

Problema: o resultado depende inteiramente do pressuposto. Se incluirmos MULTIPOP(k) com k escolhido uniformemente entre 0 e o tamanho atual s da pilha, o custo esperado de uma chamada passa a ser E[k] = s/2 — podendo voltar a ser Θ(n).

EXERCÍCIO GUIADO

Passo 3 - Análise Amortizada

Sem qualquer suposição sobre a sequência: cada elemento só pode ser empilhado uma vez antes de ser desempilhado (por POP ou MULTIPOP).

Logo, em qualquer sequência de n operações, o número total de remoções (em todas as chamadas de POP e MULTIPOP juntas) não pode exceder o número total de inserções, que é no máximo n.

$$\text{custo total} \leq n \text{ (PUSHes)} + n \text{ (remoções)} = 2n$$

Custo amortizado por operação = 2n / n = O(1)

EXERCÍCIO GUIADO

Comparando as Três Análises

Pior caso (1 operação)	$\Theta(n)$	correto, mas superestima o custo total se somado ingenuamente
Caso médio (1 operação)	$O(1)$ ou $\Theta(n)$	depende inteiramente do pressuposto assumido sobre a distribuição
Amortizada (sequência)	$O(1)$	garantida, sem suposições, via método do agregado

Conclusão: as três análises respondem perguntas diferentes, nenhuma substitui as outras, e a escolha depende do que se quer garantir.

Questões?